

2-CSP BackTracking

约束满足问题/CSP

csp constraint satisfaction problem (比如地图上色、N-Queen问题)

这是CS188课程第二部分，属于identification/assignment类的问题，和之前的planning不一样的地方是：

- 状态是一组变量，有确定的值域来给变量赋值
- 关心目标状态各个变量的具体赋值情况，而无所谓路径（赋值顺序无关）
- 有一组约束条件，状态只要全部变量都赋值且符合条件即可（完整性和一致性）

另外，以下涉及的都是二元CSP问题，这里的“二元”不是说值域是个布尔类型，指的是一个约束条件只限制**两个**变量之间的关系

比如在地图着色问题中，基本的规则是：“相邻的两个区域不能染相同的颜色”

如果我们把每个区域看作一个变量（例如

$$X_1, X_2, \dots$$

），那么每一条规则都只涉及两个变量，即

$$X_i \neq X_j$$

如果一个问题中存在一个约束条件同时涉及三个或更多变量（例如：“区域A、区域B和区域C的颜色必须互不相同”或者“区域A、B、C的颜色之和必须为10”），那么这就是一个多元约束

参考资料：[知乎](#)

回溯搜索/Backtracking Search

关于地图上色问题，一个简单的想法是基于DFS[1-搜索算法](#)，给一个个节点涂色（赋值），完成图后再检查是否符合，不符合就撤回重来

这样的效率是很低的，应当：

- 每次赋值后，就检查是否符合，不符合回溯
- 一次只赋值一个变量，变量的赋值顺序不影响结果（不关心顺序）

具备这两个特点的DFS算法叫回溯搜索，大约可以求解N~25的N-Queen问题

```
def backtrack(assignment):  
    #伪代码  
    if assignment is complete: return assignment #因为每次赋值后就检查，所以完成全部赋值必然符合约束  
    var = select_unassigned_variable() #优化
```

```

for value in order_domain_values(var): #优化
    if value is consistent with assignment:
        assignment[var] = value
        result = backtrack(assignment) #递归
        if result is not None: return result
        del assignment[var] # 回溯
return None

```

回溯搜索的优化方法

ordering

选择什么变量赋值
从值域里赋什么值

filter

提前避开可能的错误（剪枝）

structure

利用数据结构

filter: forward checking 前向检查

在涂色问题里，当一个节点涂色后，相邻节点就少了可涂的颜色，可以直接不考虑

当给变量 X 赋值后，立即从 neighbor variable 的可选值域中删除冲突值。若某个未赋值变量值域变空，说明目前的赋值模式已经失败，可以直接开始回溯

将信息从赋值的节点传播到没赋值的节点，但是不能检查出没赋值节点之间的错误

filter: arc consistency/2-consistency/AC-3 弧一致性/弧相容性

不同节点之间链接一条单向弧，head为被指向的，tail是发起箭头的，比如A -> B

对于A/tail的每一个可能取值，检查B/head的取值能不能符合规则，如果不能就说明这条弧不满足一致性（符合规则），要**从tail中删除对应的取值**

一直删到某个节点无值可赋（出现空域），就该回溯了

具体操作：将题目中所有的双向约束（弧）都放入队列中。例如，如果A和B之间有约束，则放入(A,B)和(B,A)

```

while queue not empty:
    (Xi, Xj) = queue.pop() # 取出一个弧

    # 检查Xi的值域是否需要缩减
    if revise(Xi, Xj):
        if domain[Xi] is empty:

```

```
return False # 检测到无解
```

```
# Xi的值域被缩减，需重新检查与Xi相关的其他弧，因为被删掉的值可能就支撑着某些弧的一致性
```

```
for each Xk in neighbors[Xi] except Xj:  
    queue.push((Xk, Xi)) #队列的FIFO作用，发生变化之后立即加入和邻居间的弧
```

```
revise(Xi, Xj):  
    removed = false  
    for each value x in domain[Xi]:  
        # 检查x是否在Xj的域中有相容值  
        compatible = false  
        for each value y in domain[Xj]:  
            if (x,y) satisfies constraint(Xi,Xj):  
                compatible = true  
                break  
  
        if not compatible:  
            remove x from domain[Xi]  
            removed = true  
  
    return removed
```

前向检查实际上就是只对刚刚赋值的节点与相邻节点之间的弧一致性检查

前向检查只有在赋值的时候，约束信息才会传播开来（也只到邻居），而弧一致性能够传播到更远的地方

可以作为预处理，或者和前向检查一样，在每次赋值后进行

缺点：

- 每当发生赋值，或者tail可能的值被删除的时候，该节点的邻居就要进行弧检查，从而邻居的邻居可能也要。如果检查所有弧，时间代价可能很大
- 当图不满足弧一致性，一定不是解；但是如果满足，也不一定是解，比如

节点是个三角形的地图上色，只有三种颜色

变量：A, B, C

域：所有变量 = {红, 蓝, 绿}

约束：A≠B, B≠C, C≠A

AC-3处理：

A→B A的RGB三种取值，B可以用GB RB RG对应，其他也都如此，满足弧一致性

但实际无解：只有三种颜色，无法使三个变量两两不同

ordering: Minimum Remaining Values/MRV 最小剩余值

也叫“失败优先 (fail-first-ordering)”启发式

优先选择合法取值最少的**未赋值变量**

最受限的变量最容易导致失败，而所有变量都需要赋值，早失败早回溯，不然执行到某一步再回溯代价是指数级上升的

ordering: Least Constraining Value/LCV 最少约束值

对选定变量，优先尝试对其他变量限制最少的**值**

虽然所有变量都要赋值，但是不是每个值都要用到，这样可以给之后留出足够的选择空间

more filter: k-consistency

之前介绍了弧一致性，涉及两个节点，也说明了对于涉及两个以上的节点，就不保证正确

这种方法可以扩展到更多节点的检查，即k-consistency

arc consistency 也称作 2-consistency, 3-consistency也称作path consistency

3-consistency, 就涉及两个节点和一个被检查的节点，对于两个节点的所有赋值可能，第三个节点都满足一致性

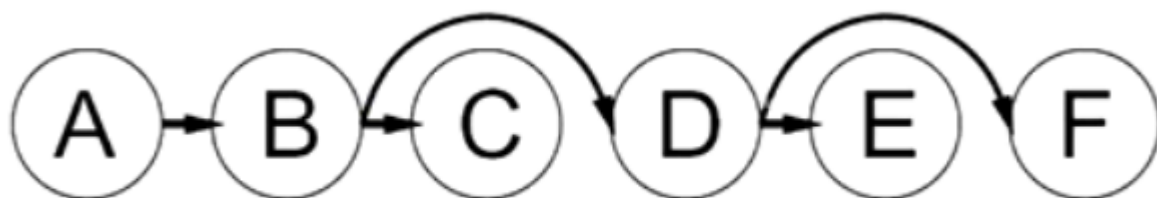
strong k-consistency同时包含了k,k-1,k-2,...,1一致性，我们可以断言它能彻底消除回溯，但是CSP问题很多都是NPC问题，随着k增加，计算代价会迅速增加

structure

这部分强烈建议去看原视频，逐过程讲解比文字描述清楚多了

在涂色问题里，如果有像岛一样的**独立单元**，解决问题将会非常方便，因为不受约束限制但事实上，这种情况不怎么出现，因为CSP问题就是约束满足问题，这些独立单元往往在问题形式化的时候就删去了

不过如果节点是**树状的**（没有环，所有子节点只有一个父节点），复杂度可以降到n的线性关系， $O(nd^2)$

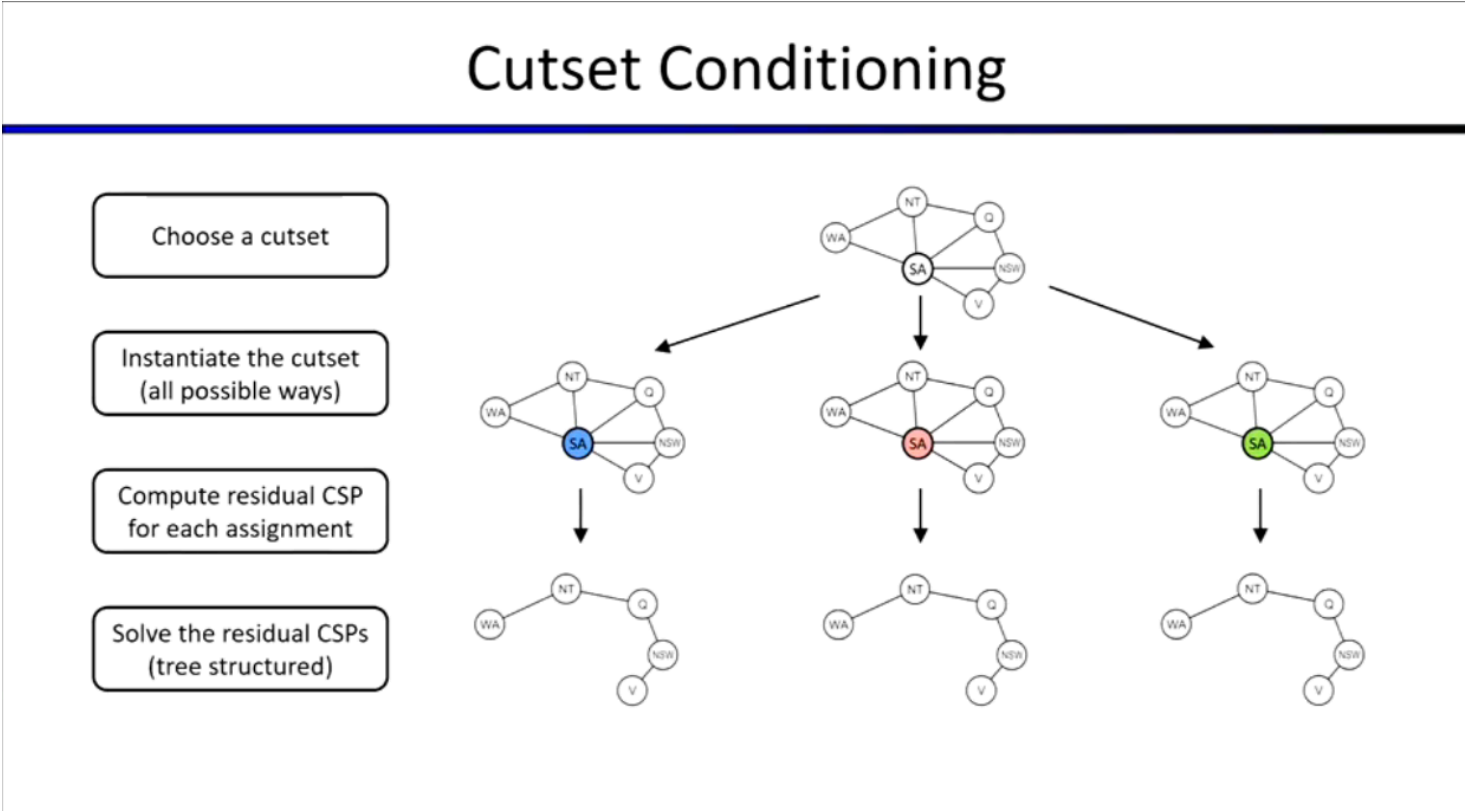


先进行反向的弧一致性遍历，从D->F D->E B->D B->C...按着头节点逆序之后再正向赋值，从A B C...赋值

刚刚的弧一致性使得：对A随便赋值，B一定有正确的赋值，从而C也有...因为所有子节点只有一个父节点，过程中不需要回溯

如果有两个父节点，比如 $A \rightarrow C \leftarrow B$
对于C来说，就算满足 $A \rightarrow C$ 和 $B \rightarrow C$ 的一致性（2-consistency），也不能确保同时满足A和B对C的一致性（3-consistency）

完全的树状结构还是很罕见，但是我们可以通过删掉某些节点，让剩余部分变成树状结构



这个例子里，SA如果确定，剩余的部分就是一个树状结构，可以迅速解决。因为不知道SA怎么赋值，每个可能的值都要试一次，然后处理剩余的树状结构

割集调节cutset conditioning的复杂度是 $O(d^c (n-c)d^2)$ ，c是去除节点的数量，c小的时候，效果很好

局部搜索

这是另一种解决CSP问题的方法

特性	系统性搜索 (如回溯法)	局部搜索 (如最小冲突算法)
状态形式	部分赋值。一次只给一个变量赋值，逐步构建解。	完全赋值。一开始就给所有变量赋值（即使违反约束）。
搜索路径	在搜索树中深度优先遍历，遇到冲突就回溯。	在解空间中移动，从一个完整状态跳到相邻状态。
目标	找到满足所有约束的精确解（完备性）。	找到一个满足约束的解，或者尽可能减少冲突（不完备）。

特性	系统性搜索 (如回溯法)	局部搜索 (如最小冲突算法)
内存消耗	较高（需要保存搜索路径）。	极低（通常只需要保存当前状态）

但是容易陷入局部最优